Requirements for SEV-SNP tests and deployments

Here are instructions for each of up to five computer roles running the Certifier Framework supporting real sev-snp VMs. In our automated tests, in `$CERTIFIER_ROOT/vm_model_tools/examples/scenario1/run-test.sh`, all five roles operate on a single SEV-SNP computer, but this is not how real deployments are factored. The instructions below descript the general case where there are five seperate machine roles.

Role 1: A typical target (deployed) SEV machine

We need to collect some information on a typical target machine to proceed. On a typical such machine:

1. Make sure SEV is enabled and the machine is "registered."
2. Get the ARK Certificate for this family of machines. You can get the ARK, ASK and VCEK certs by calling the snpguest utility supplied by AMD.

```
snpguest certificates der $CERTS_DIR
```

3. Once you've done that, the ARK certificate should be in the directory you specified in that command ($CERTS_DIR). The ARK cert is needed for the Certifier Server policy generation. All three certs are needed on each target machine.
4. Obtain machine information about SEV-SNP versions by running the utility, cupid.exe in the $CERTIFIER_ROOT/utilities directory. Save this for obtaining information needed in subsequent steps (like cpu-type, stepping, …).

Role 2: The machine that builds the VM

1. The certifier framework as built by the scripts
2. We support two types of tests. For fully automated tests ("`run-test.sh`"), there is a subscript in `$CERTIFIER_ROOT/vm_model_tools/examples/scenario1` called `build-vm.sh` and a companion script, `copy-vm-files.sh`, together these build the appropriate vmlinuz and initrd.img files for the tests. If you are running the automated tests on an sev VM as implemented in `run-test.sh`, where the build, target, and policy server all run in the same VM, you simply call that script. If not, there are separate instructions in the scenario1 example that describe alternate test and deployment scenarios as well as provided some supporting shell script examples. Some

alternatives, for example, on AWS, omit building a custom VM and use prebuilt VMs. For Datica tools (in a separate repository, called DexTools), there is a set of scripts that build VMs and integrity protected file systems that work in a slightly different way. However, because of these differences, you should know the following. For tests and deployments:

   a. At a minimum, you need a vmlinuz. The vmlinuz should have the sev device driver.
   b. At a minimum,you need an initrd.img containing cf_utility.exe, cf_key_server.exe and cf_key_server.exe together with the libraries they depend on. Ideally, as in run-test.sh, there should be an automatic build corresponding to vmlinuz naming the Linux machine policy file (e.g.-firewall rules, module loading, passwords,…) and other files to be included in initrd.img. Initram.img must also have the policy key certificate for every security domain the VM will participate in.
   c. Directories for running the Certifier in the VM.

Role 3: The machine that measures VM (often the Certifier Server)

1. A copy of the IBM measurement tool
2. The correct ovmf (firmware) file and its file location on this machine.
3. A copy of the VM elements:
   a. vmlinuz
   b. initrd.img
4. The correct arguments for target machine to be certified:
   a. cpu type
   b. number of vcpus
   c. cpu model
   d. cpu stepping
   e. cpu family
   f. cpu features
   g. vmm-type
   h. The command line for launching the VM.

Role 4: The Certifier Server (for policy building and certification)

1. The Certifier repository
2. The ARK Cert for the families to be certified
3. For each VM measurement to be certified
   a. The SEV features to be checked in validating an attestation
   b. The output of the measurement tool (usually hex) for the VM to be certified

Role 5: The target VM

1. SEV must be enabled
2. The location of the files containing the ARK, ASK and VCEK must be supplied to the certifier installation scripts.
3. The appropriate firmware (OVMF file).
4. You must have a VMM (e.g.-KVM) that supports Sev instruction set extensions. Ideally, this VMM is measured and becomes part of the certification evidence to protect against side channel attacks mounted by a VMM that stops VM execution between each instruction.
5. You must have a mechanism to launch the VM under your VMM on the target machine. This is VMM specific. Our examples do this for KVM but your milage may vary on other platforms.